ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
ΙΔΡΥΘΕΝ ΤΟ 1837

Παράλληλα Συστήματα - Εργασία 2

Ιωάννης Γρηπιώτης (sdi2100028)
Θεμιστοκλής Παπαθεοφάνους (sdi2100227)

Όλες οι ασκήσεις εκτελέστηκαν πάνω στο σύστημα linux02.di.uoa.gr του Πανεπιστημίου Αθηνών, το οποίο διαθέτει:

- επεξεργαστή Intel Core i7-6700 4c/8t @ 3.40GHz,
- 16GB μνήμη RAM,
- λειτουργικό σύστημα Ubuntu 20.04.6 LTS,
- έκδοση πυρήνα 5.4.0-216-generic,
- έκδοση μεταγλωττιστή GCC 9.4.0

Κάθε άσκηση εκτελέστηκε 10 φορές με την μέση τιμή του χρόνου εκτέλεσης να υπολογίζεται από το ίδιο το εκτελέσιμο.

Μετά την εκτελέση make κάθε benchmark εκτελείται από το παρακάτω script ως εξής:

```
./scripts/run_<exercise_name>.sh
```

όπου το `<exercise_name>` είναι το όνομα της άσκησης που πρόκειται να εκτελεστεί.

ΑΣΚΗΣΗ 1.1

Περιγραφή Προβλήματος

Γινόμενο 2 πλήρων πολυωνύμων μεγάλου βαθμού χρησιμοποιώντας OpenMP για την παραλληλοποίηση.

Δημιουργία πολυωνύμων

Χρησιμοποιήθηκαν threads, όσα ήταν στη διάθεση, προκειμένου να γίνει όσο πιο γρήγορα γίνεται η αρχικοποίηση. (**κέρδος** 1 τάξη μεγέθους σε sec)

Υλοποίηση γινομένου

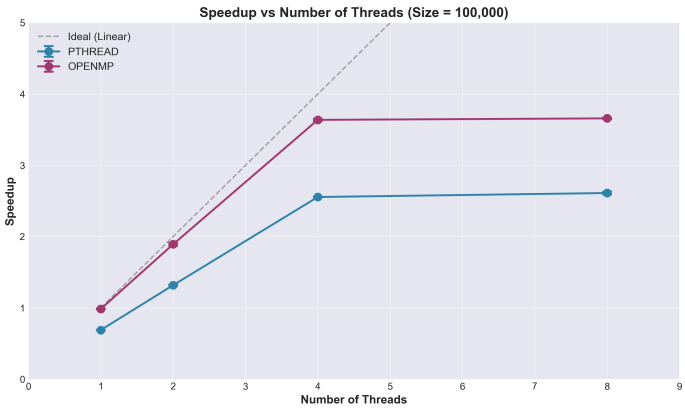
Χωρίστηκε το 1 από τα 2 πολυώνυμα σε ίσα μέρη ανάλογα με τον αριθμό των threads και έπειτα το κάθε thread έκανε το γινόμενο των 2 πολυωνύμων σε έναν δικό του μοναδικό χώρο. Στο τέλος όλα τα threads ένωσαν τα αποτελέσματά τους σε έναν τελικό ενιαίο χώρο.

Παρατηρήσεις

Η υλοποίηση του προβλήματος με τη χρήση OpenMP ήταν αρκετά πιο απλή σε σχέση με την αντίστοιχη σε pthreads απαιτώντας πολύ λιγότερες γραμμές κώδικα.

Αποτελέσματα

Benchmarks για μεγέθη 1K, 10K, 100K συντελεστών με 1-16 threads.



Μέγεθος 100K:

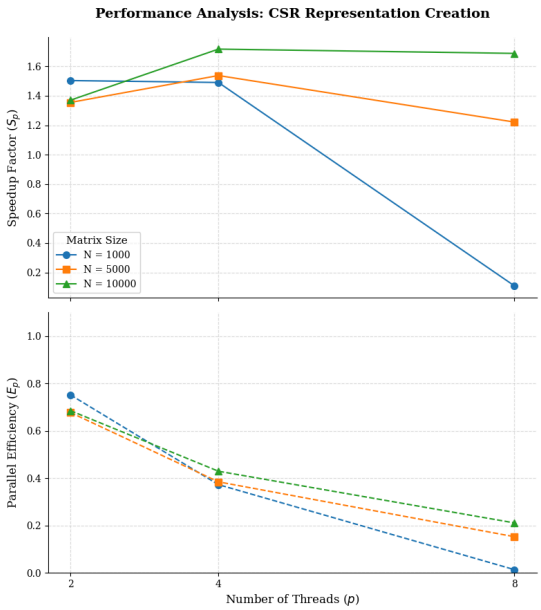
OpenMP επιτυγχάνει μέγιστο speedup 3.67× (16 threads, αποδοτικότητα 23%) έναντι 2.73× για pthread (αποδοτικότητα 17%). Το OpenMP φτάνει στο plateau στα 4 threads (3.64×, αποδοτικότητα 91%), συμπίπτοντας με τους φυσικούς πυρήνες του επεξεργαστή (4c/8t). Η αύξηση threads (8-16) δίνει οριακό κέρδος λόγω hyperthreading overhead και memory bandwidth bottleneck.

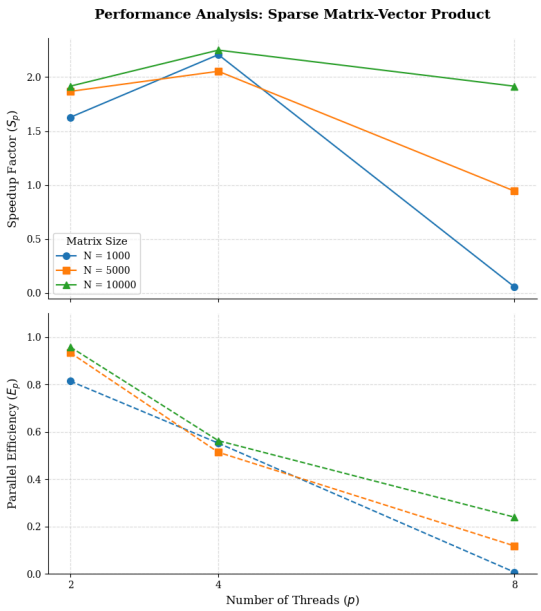
Το OpenMP υπερτερεί κατά 25-34% σε speedup για τον ίδιο αριθμό threads. Για μικρότερα μεγέθη (1K) το OpenMP φτάνει 4.87× speedup στα 8 threads, δείχνοντας καλύτερη επεκτασιμότητα σε μικρά datasets σε σχέση με μικροτέρα και φυσικά καλυτερη απόδοση απο την αντίστοιχη υλοποίηση με pthreads.

ΑΣΚΗΣΗ 1.2

Κατασκευη και Γινομενο CSR-Vector

Τα παρακάτω πειράματα έγιναν με **Sparsity** = 0%. Και με 1 μόνο επανάληψη γινομένου Πίνακα-Διανύσματος, μιας και το **scalability** δεν επηρεάζεται από τον αριθμό των επαναλήψεων, παρά μόνο από το **μέγεθος του πίνακα-διανύσματος** και τον **αριθμό των threads**.





Size	Threads	Create (Sup)	Create (Eff)	Prod (Sup)	Prod (Eff)
10^3	2	x1.504	0.752	x1.628	0.814
10^3	4	x1.491	0.373	x2.208	0.552
10^3	8	x0.109	0.014	x0.058	0.007
$5 * 10^3$	2	x1.355	0.677	x1.868	0.934
$5 * 10^3$	4	x1.538	0.384	x2.054	0.514
$5 * 10^3$	8	x1.222	0.153	x0.945	0.118
10^4	2	x1.370	0.685	x1.916	0.958
10^4	4	x1.717	0.429	x2.250	0.563
10^4	8	x1.688	0.211	x1.916	0.239

Συμπερασμα & Παρατηρησεις

Όπως φαίνεται αναλυτικά στον πίνακα και στα διαγράμματα, μπορεί κανείς να παρατηρήσει ότι η παράλληλη δημιουργία και το παράλληλο γινόμενο έχουν παρόμοια συμπεριφορά, όσο αφορά το **Speed Up** και το **Efficiency**. Τόσο όσο αυξάνονται τα **threads** όσο και όταν αυξάνεται το μέγεθος εισόδου.

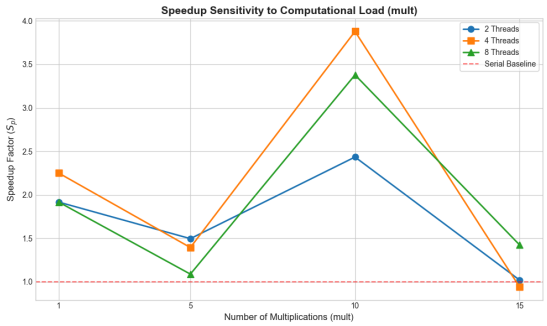
Ακόμα παρατηρείται ότι όσο μεγαλώνει το μέγεθος εισόδου για σταθερό αριθμό **threads** τόσο πιο πολύ αυξάνεται το **Speed up** και το **Efficiency**. Αν είχαν γίνει πειράματα με ακόμα μεγαλύτερο αριθμό εισόδου θα υπήρχε με βεβαιότητα ακόμα μεγαλύτερο **Speed up** από αυτό που έχουμε πχ στα **8 threads με μέγεθος εισόδου 10^4**. Αυτή η τάση κλιμάκωσης μπορεί να φανεί αρκετά ξεκάθαρα κυρίως στη δημιουργία ,μιας και, στο γενόμενο υπάρχει αλλά με πιο ήπια συμπεριφορά.

Ο λόγος για τον οποίο παρατηρείται η μείωση της απόδοσης από 4 σε 8 **threads** βασίζεται ελάχιστα στα **memory boundaries** όπως είναι το **ping-pong effect** και κυρίως στη **run-time** υλοποίηση της **OpenMP** μιας και η συνεχής δημιουργία, διαγραφή και σωστός **συγχρονισμός-barrier** που είναι απαραίτητα για τη σωστή λειτουργία προσδίδουν αρκετή καθυστέρηση. Είναι φανερό ότι αυτή η καθυστέρηση δεν μπορεί να καλυφθεί για τόσο μικρά μεγέθη από την εργασία που πρέπει να γίνει, είτε αυτό είναι **create** είτε είναι **product**.

Λαμβάνοντας τα παραπάνω υπόψη σαν ιδανική επιλογή προτιμάται τόσο η υλοποίηση με τα 2 όσο και με τα 4 **threads**. Μιας και στα 4 είχαμε το καλύτερο **Speed up** χάνοντας όμως σημαντικά το **Efficiency** σε αντίθεση με τα 2 **threads**. Επομένως αναλόγως με τους πόρους που έχει κανείς μπορεί να διαλέξει κατάλληλες παραμέτρους μεγιστοποιώντας την απόδοσή του.

Πειραματα με επαναληψεις γινομενου

Μέγεθος = 10^4

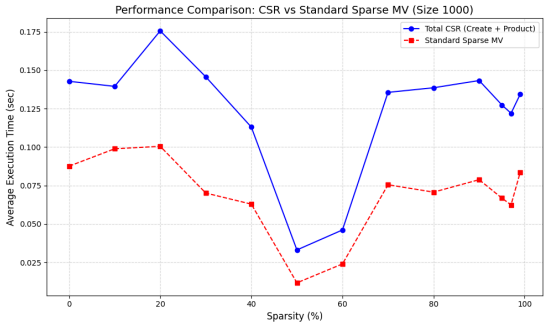


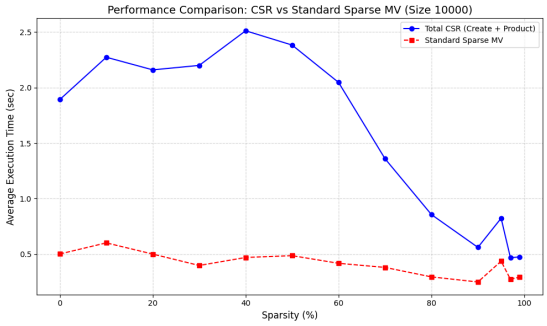
Parallel CSR-Vector VS Parallel Matrix-Vector

Με βάσει τα προηγούμενα πειράματα κρατήθηκε σαν βέλτιστη επιλογή **threads = 4**. Ως εκ τούτου έγιναν πειράματα προκειμένου να ελεγχθεί η συμπεριφορά των χρόνων όσο αφορά τη μεταβολή του **Sparsity**, αλλά και όσο αφορά κάποιες τιμές του **Multiplication times**.

Sparsity

Πειράματα έγιναν σε μέγεθος πίνακα-διανύσματος = 10^3 και 10^4, κρατώντας τον **Αριθμό επαναλήψεων γινόμενου** σταθερό και ίσο με 1. Τα αποτελέσματα είναι σε **[sec]**





Sparsity	CSR10^3	MV 10^3	CSR 10^4	MV 10^4
0	0.142	0.087	1.892	0.501
10	0.139	0.098	2.273	0.601
20	0.175	0.100	2.159	0.500
30	0.145	0.070	2.199	0.398
40	0.112	0.062	2.511	0.470
50	0.033	0.011	2.382	0.486
60	0.046	0.023	2.046	0.417
70	0.135	0.075	1.359	0.380
80	0.138	0.070	0.856	0.295
90	0.143	0.078	0.561	0.249
95	0.127	0.066	0.823	0.436
97	0.122	0.062	0.467	0.274
99	0.134	0.083	0.475	0.293

Συμπερασμα Sparsity

Όπως μπορεί να παρατηρηθεί παραπάνω, όταν υπάρχει μόνο μια επανάληψη δεν αξίζει η δημιουργία και η χρήση της **CSR** εναντη της χρήσης της **Dense** αναπαράστασης. Αυτό είναι σχετικά αναμενόμενο μιας και ξοδεύονται αρκετοί πόροι προκειμένου να δημιουργηθεί μια δομή που θα χρησιμοποιηθεί μόνο μια φορά. Αυτόν τον χρόνο τον αξιοποιεί η **Dense** αναπαράσταση αρχίζοντας άμεσα τους υπολογισμούς του γινόμενου.

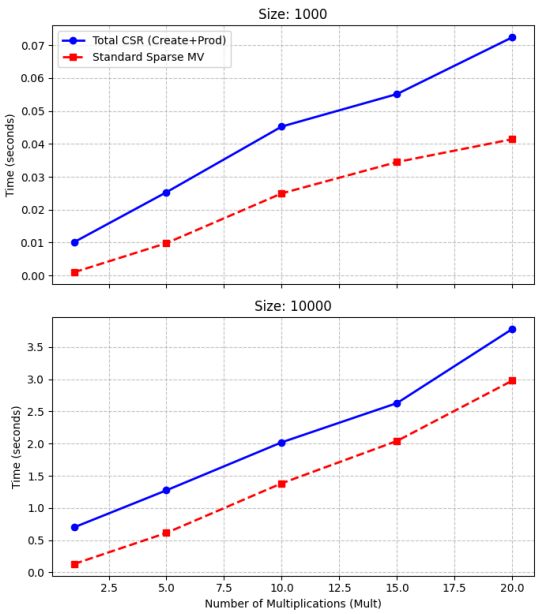
Κάτι ακόμα που έχει ενδιαφέρον είναι η σχετικά παρόμοια συμπεριφορά όσο αφορά το 10^3 μέγεθος κατά τη διάρκεια όλης της μεταβολής του **Sparsity**. Αντίθετα αυτό παρατηρείται μόνο για 90% και πάνω στα πειράματα με το 10^4 μέγεθος.

Επομένως, αν έπρεπε να γινει ένα απλό γινόμενο (**multiplication times** = 1) πίνακα-διανύσματος, η χρήση **Dense** αναπαράστασης είναι **μονόδρομος**.

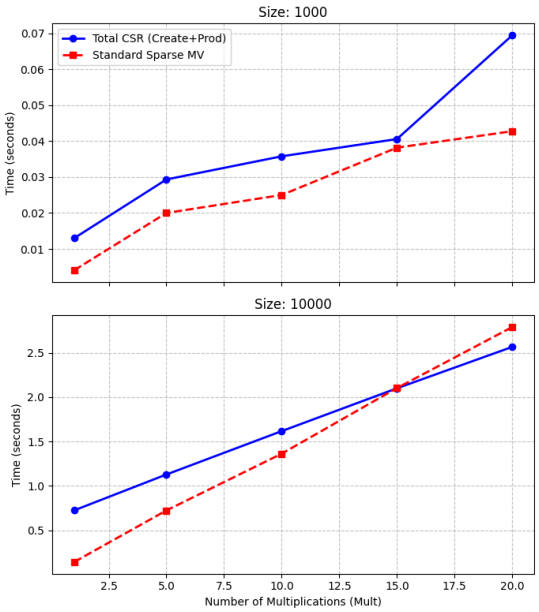
Multiplication times

Όπως παρατηρήθηκε νωρίτερα, η παράλληλη δημιουργία και υλοποίηση του γινόμενου **CSR-vector** δεν μπόρεσαν να κάνουν **overlap** αυτο της **Dense** αναπαράστασης. Αυτό δεν σημαίνει ότι η **CSR** αναπαράσταση είναι κακή ή ακατάλληλη για πρακτικές εφαρμογές. Προκειμένου να αποδειχθεί αυτό, έγιναν κάποια πειράματα για κάποιες, "ενδιαφέρουσες" με βάσει τα προηγούμενα 2 διαγράμματα, τιμές του **Sparsity**. Πιο συγκεκριμένα για 25%, 50%, 95%, 99%. Δηλαδή τιμές οι οποίες έδειξαν ότι συγκλίνουν τους χρόνους ή ότι δημιουργούν απόλυτο μέγιστο-ελάχιστο.

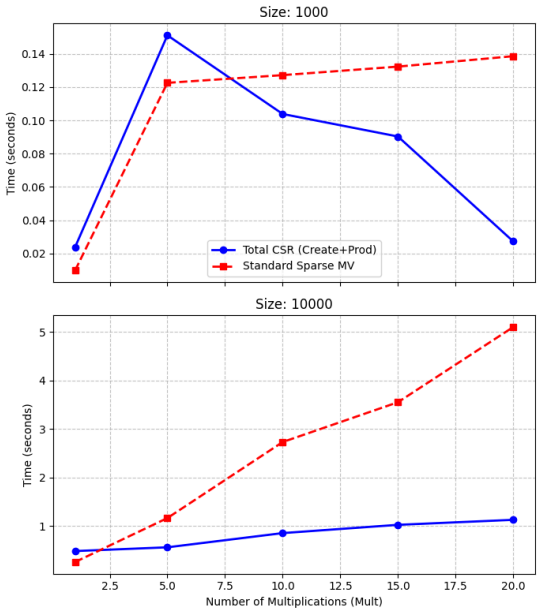
Performance Comparison - Sparsity 25%



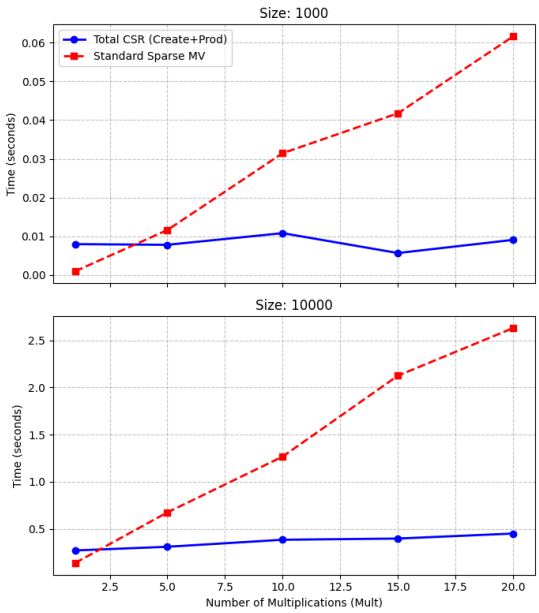
Performance Comparison - Sparsity 50%



Performance Comparison - Sparsity 95%



Performance Comparison - Sparsity 97%



Συμπερασμα & Παρατηρησεις

Όπως παρατηρείται υπάρχουν ανάμεικτα αποτελέσματα και συμπεριφορές. Πιο συγκεκριμένα για **Sparsity** 25% ακόμα και με μεγάλο αριθμό επαναλήψεων υπηρχε κυριαρχία της **Dense** έναντι της **CSR** αναπαράστασης. Το ίδιο φαινόμενο παρατηρείται και για μέγεθος 10^3 με **Sparsity** 50%, αντίθετα για μέγεθος 10^4 υπάρχει συνεχής σύγκλιση των χρόνων καθώς αυξάνεται ο αριθμός των επαναλήψεων έχοντας αλλαγή κυριαρχίας από την **CSR** αναπαράσταση για 15 και πάνω (επαναλήψεις). Τέλος όσο αφορά το **Sparsity** 95% και 99% και τα δύο μεγέθη έχουν την ίδια συμπεριφορά, με μια μικρή αντίσταση όσο αφορά το μέγεθος 10^3, παρατηρώντας κυριαρχία της **CSR** αναπαράστασης από τις πρώτες κιόλας επαναλήψεις.

Κάποιες συμπεριφορές χρόνων ήταν κάπως αναμενόμενες. Πιο συγκεκριμένα η **CSR** αναπαράσταση μπορεί να κυριαρχήσει σε μεγάλα **Sparsity** ακόμα

και με μικρό μέγεθος διότι λόγω της δομής γίνονται πολύ λιγότερες πράξεις από την πυκνή. Ακόμα με μέτριο βαθμό **Sparsity** αν υπάρχει αρκετός αριθμός επαναλήψεων η υλοποίηση της **CSR** γίνεται πιο γρήγορη από την **Dense** διότι για κάθε επανάληψη προλαβαίνει να "μαζέψει" τον χρόνο που σπαταλήθηκε προκειμένου να δημιουργηθεί. Αντίθετα για μικρά **Sparsity** δεν προλαβαίνει να αλλάξει αυτή η διαφορά. Ακόμα όσο πιο αραιός είναι ένας πίνακας τόσο πιο **απαραίτητη** και **κερδοφόρα** θα είναι η χρήση μιας **CSR** αναπαράστασης. Ο τελευταίος ισχυρισμός αποτελεί κύριο κλειδί της αποδοτικής χρήσης του.

Επομένως, όπως παρατηρήθηκε από όλα τα παραπάνω πειράματα, δεν μπορεί κάποιος να αποφασίσει με βεβαιότητα εκ των προτέρων ποια από τις 2 αναπαραστάσεις είναι η καλύτερη. Αυτό οφείλεται στο γεγονός ότι η σύγκριση δεν επηρεάζεται από έναν και μοναδικό παράγοντα, αλλά από σύνολο αυτών. Πιο συγκεκριμένα επηρεάζεται από το **μέγεθος**, το **Sparsity** (ποσοστό μηδενικών), τον αριθμό των **επαναλήψεων γινόμενου** ακόμα και τον αριθμό των **threads**.

Γενικα

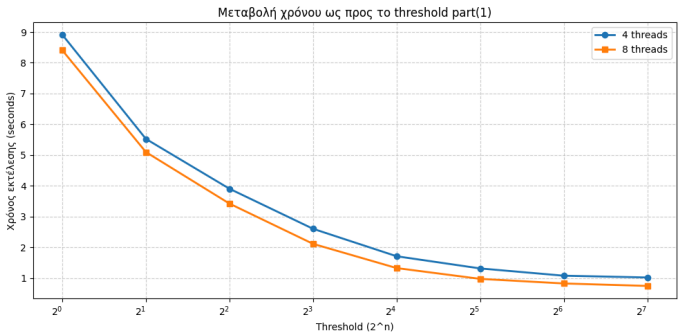
Στην υλοποίηση των παράλληλων αλγορίθμων υπάρχει σχετικά μικρή καθυστέρηση λόγω των γραμμικών κομματιών και των σημείων **συγχρονισμού-integrated barrier** προκειμένου να υπάρξει σωστό αποτέλεσμα.

Ακόμα όπου έγινε παράλληλη η **for (#pragma omp for)** χρησιμοποιήθηκε ο **default scheduler**, μειώνοντας όσο γίνεται τη **run-time** καθυστέρηση που θα προκαλούσε η **OpenMP** για καταχώρηση βάρους εργασίας.

ΑΣΚΗΣΗ 1.3

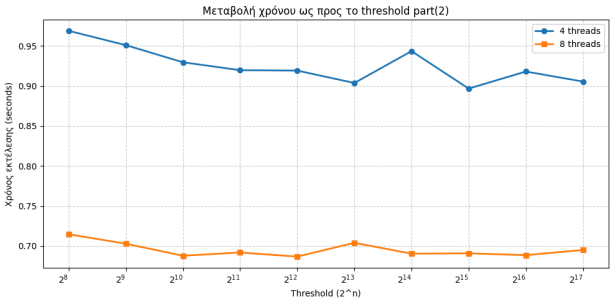
Ερευνα βελτιστου Threshold

Έγιναν κάποια πειράματα προκειμένου να βρεθεί αν όχι η βέλτιστη, κάποια αρκετά καλή τιμή **threshold** του **if clause** που δηλώνει αν ένα **OpenMP task** θα δημιουργηθεί ή όχι. Δηλαδή αν θα γίνει παράλληλα ή σειριακά. Αυτά τα πειράματα έγιναν σε αρκετά μεγάλο μέγεθος πίνακα της τάξης 10^8. Αργότερα θα αποδειχτεί εξαιρετικά **tricky** σαν επιλογή για κάποια αποτελέσματα πειραμάτων.



Τα αποτελέσματα είναι σε **[sec]**.

Threshold	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7
4 threads	8.91	5.52	3.90	2.60	1.71	1.32	1.08	1.02
8 threads	8.40	5.10	3.42	2.11	1.33	0.98	0.83	0.75



Τα αποτελέσματα είναι σε **[sec]**. 1η στήλη: **Threshold**, 2η: **4 threads**, 3η: **8 threads**

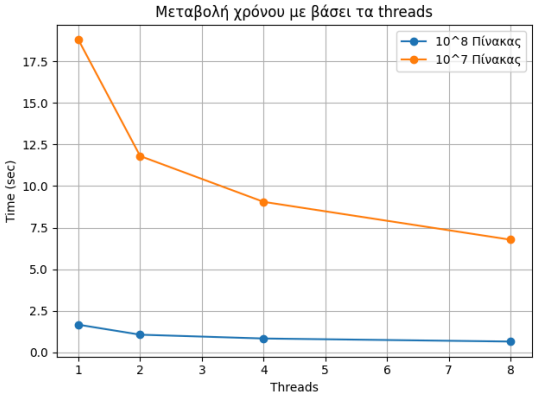
2^8	2^9	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}	2^{17}
0.9689	0.951	0.9296	0.9197	0.9192	0.9037	0.9435	0.8967	0.918	0.9055
0.7146	0.7028	0.6878	0.6918	0.6867	0.7039	0.6904	0.6908	0.6885	0.695

Συμπερασμα

Ο σκοπός του προβλήματος είναι η ταξινόμηση ενός μονοδιάστατου πίνακα ακεραίων (**int**). Στο σύστημα που έγινε η υλοποίηση το μέγεθος τους αποτελείται από **4 bytes**. Ακόμα ένα **cache line** έχει μέγεθος **64 bytes**. Εξαιτίας των παραπάνω είναι λογικό για **threshold** μέχρι και **16** να υπάρχει μεγάλη καθυστέρηση χρόνου λόγω **false sharing** δηλαδή **ping-pong effect**. Παρατηρείται συνεχής κάθοδος χρόνου μέχρι το $2^{\{10\}}$ έχοντας έπειτα κάποιες αυξομειώσεις. Αν και δεν είναι μεγάλες οι διαφορές παρατηρούνται ως πιο σταθερά μεγέθη τα $2^{\{10\}}$, $2^{\{11\}}$ και $2^{\{12\}}$. Κρατώντας ως ιδανικό, **threshold**, το **τελευταίο** μέγεθος ($2^{\{12\}}$) χωρίς βλάβη της γενικότητας.

Γενικά για πολύ μικρές τιμές **threshold** εκτός από **false sharing** υπάρχει και ένα άλλο φαινόμενο που επηρεάζει τον χρόνο. Αυτό το φαινόμενο ονομάζεται **overhead** και ουσιαστικά γίνεται όταν η "εργασία" που πρέπει να γίνει στο **task** είναι $\ll$ από τον χρόνο που χρειάζεται για να δημιουργηθεί και να ανατεθεί σε ένα **thread**.

Speed Up και Efficiency



Αποτελέσματα σε **[sec]**. Με **Sp-Up** = Speed Up και **Eff** = Efficiency

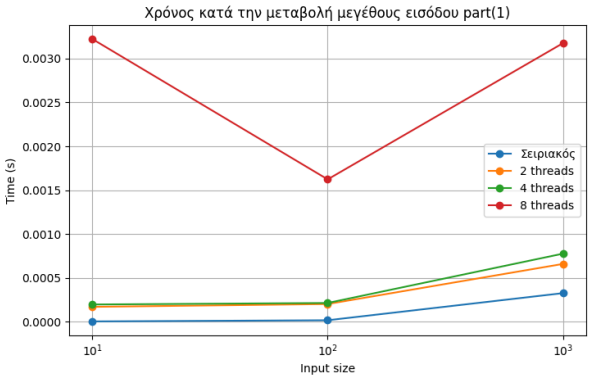
Threads	**Time (10^7)**	**Sp-Up (10^7)**	**Eff (10^7)**	**Time (10^8)**	**Sp-Up (10^8)**	**Eff (10^8)**
1	1.662	x1	100%	18.794	x1	100%
2	1.063	x1.56	78%	11.805	x1.59	79.5%
4	0.833	x1.99	49.7%	9.050	x2.07	51.7%
8	0.651	x2.55	31.8%	6.775	x2.77	34.6%

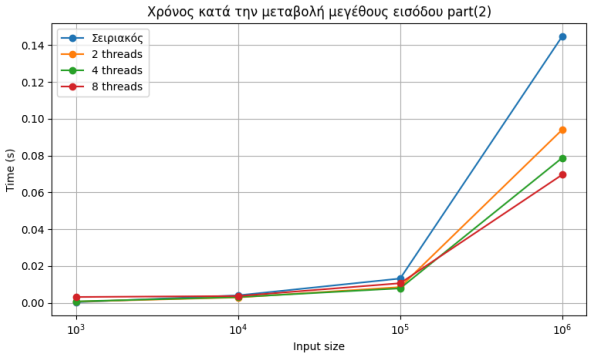
Συμπερασμα

Τα αποτελέσματα είναι αναμενόμενα όσο αφορά τις τιμές ανάμεσα στις αλλαγές μεγέθους. Παρατηρείται ότι υπάρχει θεαματικό **speed-up** έως και **x2.5** μειώνοντας όμως αρκετά το **efficiency**. Αυτό γίνεται προφανώς λόγω **run time** καθυστερήσεων της **OpenMP** όπως και κάποιων αμιγώς σειριακών κομματιών όπως πχ το να μην δημιουργούνται άλλα **tasks** μετά από ένα **threshold**. Κάνοντας **μη scalable** την εφαρμογή. Επομένως σαν βέλτιστη επιλογή είναι τα 2 και τα 4 **threads**, ειδικά αν κάποιος θέλει να αξιοποιήσει παράλληλα σε κάποια άλλη εφαρμογή αρκετούς πυρήνες.

Πειραματα

Έγιναν κάποια πειράματα από "περιέργεια" προκειμένου να εξεταστεί η συμπεριφορά του χρόνου καθώς μεταβάλλεται τόσο ο αριθμός των **threads** αλλά και το **μέγεθος του πίνακα**.





Οι παρακάτω χρόνοι είναι σε μορφή **[msec]**, εκτός αν αναφέρεται ρητά η μονάδα χρόνου. **Par** = Parallel, δηλαδή παράλληλη υλοποίηση με **x threads**.

Size	Serial	2 Threads Par	4 Threads Par	8 Threads Par
10^1	0.004	0.170	0.197	3.222
10^2	0.017	0.202	0.214	1.623
10^3	0.326	0.659	0.776	3.175
10^4	4.068	2.945	3.119	3.687
10^5	13.201	8.548	7.877	10.657
10^6	0.144**[sec]**	0.094**[sec]**	0.078**[sec]**	0.069**[sec]**

Συμπερασμα

Τα αποτελέσματα σε κάποιες από τις περιπτώσεις δεν ήταν τόσο πολύ αναμενόμενα. Πιο συγκεκριμένα όσο αφορά τα αποτελέσματα μέχρι μέγεθος 10^3 παρατηρείται κυριαρχία του σειριακού (1). Μέχρι μέγεθος 10^4 στον παράλληλο αλγόριθμο κυριαρχούν τα **2 threads** ακολουθούμενα από τα **4** και τέλος τα **8 threads** (2). Στο 10^5 εναλλάσσεται η θέση ανάμεσα στα **2** και στα **4 threads** (3). Τέλος από 10^6 κυριαρχεί ξεκάθαρα η υλοποίηση με τα **8 threads** (4).

Πιθανή αιτιολόγηση αποτελεσμάτων

(1). Ο χρόνος που χρειάζεται για να δημιουργηθούν και να καταστραφούν τα **threads** είναι πολύ μεγαλύτερος από το να γινόταν απλά **σειριακά** η ταξινόμηση.

(2). Εξαιτίας του **threshold** που έχει επιλεγεί **4096** τα **tasks** που δημιουργούνται είναι το πολύ 2. Όπως και στο (1) η δημιουργία και καταστροφή των **threads** έχει καθοριστικό ρόλο στον χρόνο. Κάνοντας τα **2 threads** την βέλτιστη επιλογή εξαιτίας των λίγων εργασιών.

(3). Όμοιο με το (2)

(4). Πλέον τα **tasks** είναι αρκετά προκειμένου να γίνει **overlap** ο σειριακός χρόνος.

Με λίγα λόγια. Ο λόγος που δεν μπορεί να κυριαρχήσει πάντα η εκτέλεση με τα **8 threads** έγκειται στα παρακάτω:

- "Κακή" επιλογή **threshold**. Αυτός ο παράγοντας καθορίζει τον αριθμό των **tasks** που θα δημιουργηθούν. Υπάρχουν πολύ λίγα **tasks** και πολλά **threads**. Αν και η τιμή είναι βέλτιστη για μεγάλο αριθμό πινάκων, δεν είναι εξίσου αξιόπιστη για μικρούς.
- **Run time** εργασία και συγχρονισμός της **OpenMP**. Το να συγχρονιστούν, να δημιουργηθούν και να καταστραφούν σωστά τα threads είναι κοστοβόρο. Ακόμα σε αυτό συμμετέχει και ο συγχρονισμός που πρέπει να γίνει για το ποια **threads** θα κάνουν ποια **tasks** (**scheduling**), όπως ακόμα και το **stealing**. Δεν έχουν όλα τα **tasks** το ίδιο φορτίο εργασίας, εξαιτίας της φύσης του αλγορίθμου.

Στον πραγματικό κόσμο όμως και σε πραγματικές εφαρμογές, υπάρχει τεράστιος όγκος από δεδομένα. Στρέφοντας το ενδιαφέρον σε βελτιστοποιήσεις τέτοιου μεγέθους και όχι μικρότερων. Παράλαυτα, το παραπάνω πείραμα διδάσκει το γεγονός ότι κάποια πράγματα δεν είναι όσο προφανή και όσο απλά θεωρούνται.

Ευχαριστούμε τον xmamalou για το template της αναφοράς.